

# 无哈希函数的哈希表设计综述

巨文博

(南京大学智能软件与工程学院 苏州 215000)

**摘 要** 哈希表通常依赖哈希函数完成键到存储位置的映射,但显式哈希函数族的能力会限制其失败概率。Kuszmaul 在论文“A Hash Table Without Hash Functions, and How to Get the Most Out of Your Random Bits”中将随机性直接嵌入数据结构,提出旋转基数树及其两种改进结构。本文梳理旋转基数树的数组旋转机制,分析增强旋转基数树如何利用溢出球和集中界降低失败概率,并说明低随机性旋转基数树如何借助分组映射减少随机比特。三类结构以相同的随机旋转框架为基础,分别体现了操作时间、失败概率、随机比特和空间开销之间的不同权衡。

**关键词** 哈希表; 随机化字典; 旋转基数树; 失败概率; 随机比特; 简洁字典

## 1 引言

字典(dictionary)维护键值对集合,并支持插入、删除和查询。随机化字典通常借助哈希函数将键映射到桶或槽位,从而在期望意义或高概率意义下实现常数时间操作。

本文综述的对象是 Kuszmaul 提出的无哈希函数哈希表构造<sup>[1]</sup>。该论文关注的问题是:在存储  $n$  个  $\Theta(\log n)$  位键值对、使用线性空间并支持常数时间操作的前提下,哈希表能够提供多强的概率保证。

传统分析通常将概率保证与哈希函数族联系在一起。完全随机函数能够提供较强的负载均衡,却难以被显式、高效地实现;常数时间可计算的显式哈希函数族又往往只能提供有限独立性,因而难以继续降低失败概率。

Kuszmaul 的处理方式是改变随机性的载体:不再直接计算  $h(x)$  形式的键到桶映射,而是为基数树的节点数组设置随机旋转量。本文依次讨论基础旋转结构、面向极低失败概率的增强结构、面向少量随机比特的低随机性结构,以及它们与简洁字典之间的联系。

## 2 研究背景

理解旋转基数树之前,需要先明确本文所采用的失败定义及传统哈希方法的限制。前者决定概率保证的含义,后者说明为何需要把随机性从哈希函数转移到数据结构内部。

### 2.1 字典与失败概率

在本文讨论的随机化字典中,失败并非返回错误结果,而是某次操作未能在常数时间内完成。当失败概率不超过  $1/\text{poly}(n)$  时,通常称该操作以高概率在常数时间内完成。相关分析还采用遗忘型对手假设,即操作序列独立于数据结构

所使用的随机比特。

确定性的常数时间字典是一个重要问题。已有工作如动态融合节点(dynamic fusion node)可以对较小规模集合实现确定性常数时间操作<sup>[2]</sup>,但对于  $\Theta(n)$  规模的一般集合,确定性常数时间字典仍然是困难问题。

### 2.2 哈希函数瓶颈

高概率哈希表通常依赖哈希函数族。完全随机哈希函数在理论上具有较强的负载均衡能力,但将其替换为可显式构造且能在常数时间求值的函数族后,概率保证往往随之减弱。

Kuszmaul 因此不再把改进重点放在哈希函数族上,而是将随机性直接放入数据结构。后文三个结构都建立在这一变化之上。

## 3 旋转基数树(Rotated Radix Trie)

旋转基数树是后续两种结构的共同基础。它先利用随机旋转压缩普通基数树的节点数组,再通过球入桶模型分析叠加后数组中的最大负载。

### 3.1 基本结构

普通基数树根据键的不同片段逐层选择子节点,能够确定性地支持常数时间操作;但若直接为每个内部节点维护长度为  $n$  的数组,最坏空间会达到二数量级。旋转基数树(rotated radix trie)通过叠加这些稀疏数组降低空间开销。

旋转基数树的做法是:为每个内部节点  $s$  选择一个随机旋转量  $r_s$ ,然后把各个节点数组经过循环旋转后叠加到一个总数组中。如果某个非空指针原本位于子节点编号  $c$ ,则旋转后映射到

$$\phi(s, c) = (c + r_s) \bmod n. \quad (1)$$

这个公式体现了该结构与传统哈希表的区别：它没有对每个键计算哈希值，而是对节点数组整体进行随机旋转。

### 3.2 球、桶与负载

为了分析该结构，论文将每个非空指针称为一个球 (ball)，将叠加后总数组中的位置称为桶 (bin)。一个球可以表示为  $(s, c)$ ，其中  $s$  是源节点， $c$  是该节点中的子节点编号。

对某个桶  $j$ ，设  $Y_j$  表示映射到该桶的球数量，则  $Y_j$  就是该桶的负载 (load)。由于总共有  $O(n)$  个球，而每个球落到固定桶的概率约为  $1/n$ ，因此可以得到

$$\mathbb{E}[Y_j] = O(1). \quad (2)$$

由于各节点的旋转量相互独立， $Y_j$  可以写成一组独立指示变量之和。Chernoff 界给出  $Y_j \leq \text{polylog}(n)$  的概率至少为  $1 - 1/n^{\text{polylog}(n)}$ 。在这一事件下，每个桶都可以用动态融合节点存储，从而维持常数时间操作。

### 3.3 结构小结

旋转基数树由此获得线性空间和略低于多项式量级的失败概率。该结构并未达到后文所需的极低失败概率或近对数随机比特，但它把随机性约化为节点旋转量，为两种改进结构提供了统一起点。

## 4 增强旋转基数树 (Amplified Rotated Trie)

增强旋转基数树沿用球到桶的映射，但允许少量球暂时离开主数组，并通过集中不等式控制这些溢出球的总数。该设计将单个桶的局部负载问题转化为辅助结构的整体空间问题。

### 4.1 溢出球 (Overflow Ball)

当某个桶已经存放  $\text{polylog}(n)$  个球，而新球仍映射到该桶时，新球被称为溢出球 (overflow ball)。增强旋转基数树 (amplified rotated trie) 不再强行把它放入已满的动态融合节点，而是将其转移到辅助结构  $Q$ 。

辅助结构  $Q$  使用  $n^\delta$  叉树实现，可以在常数时间内处理溢出球，但存储  $q$  个溢出球时最多消耗  $qn^\delta$  空间。因此，维持线性空间需要把  $q$  控制在  $n^{1-\delta}$  以下。

### 4.2 控制溢出数量

论文将溢出球总数  $q$  看成所有随机旋转量的函数：

$$f(r_1, r_2, \dots, r_m) = q. \quad (3)$$

为分析该函数的集中性，论文使用 McDiarmid 不等式<sup>[3]</sup>。将分支因子 (fanout) 从  $n$  降为  $n^\delta$  后，一个节点至多产生  $n^\delta$  个球，因而改变单个旋转量对  $q$  的影响也不超过  $n^\delta$ 。这一有界差分性质使 McDiarmid 不等式能够用于估计  $q$  偏离期望的概率。

### 4.3 概率保证

取  $\delta = \varepsilon/5$ ，上述集中界可将  $q \geq n^{1-\delta}$  的概率控制为

$$O\left(\frac{1}{n^{1-\varepsilon}}\right) \quad (4)$$

的量级。因此，增强旋转基数树在保持线性空间的同时，使每次操作以  $1 - O(1/n^{1-\varepsilon})$  的概率在常数时间内完成。对于  $\Theta(\log n)$  位键值，这一保证已接近非显式确定性常数时间字典所对应的概率界限。

## 5 低随机性旋转基数树 (Budget Rotated Trie)

低随机性旋转基数树 (budget rotated trie) 不再追求极低失败概率，而是在保持  $1/\text{poly}(n)$  失败概率的条件下减少随机比特。原论文取  $\delta = 1/4$ ，最终只需  $O(\log n \log \log n)$  个随机比特。

为了减少随机比特的使用，低随机性旋转基数树不再为大量内部节点直接生成完全独立的随机旋转量。一个基本做法是减少真正需要显式创建的内部节点数量：当某个子树规模较小时，可以先用动态融合节点直接存储，只有当子树规模超过阈值后才创建新的内部节点。

在此基础上，论文进一步改变球到桶的映射方式。基础旋转基数树使用  $\phi(s, c) = (c + r_s) \bmod n$ ，而低随机性旋转基数树将全部桶分为  $n^\delta$  个组，每组大小为  $n^{1-\delta}$ ，并定义

$$\psi(s, c) = ((c + a_s) \bmod n^\delta) \cdot n^{1-\delta} + b_s \quad (5)$$

其中  $a_s$  决定球所在的组， $b_s$  决定组内位置。 $a_s$  由常数阶独立哈希函数产生，用于控制每组接收的球数； $b_s$  由具有负载均衡性质的逐渐增强独立性哈希函数 (gradually-increasing-independence hash functions) 产生，用于限制组内最大负载。两类参数分开生成，避免了为每个内部节点保存完全独立的随机旋转量。

逐渐增强独立性哈希函数需要  $O((\log \log n)^2)$  时间求值，不能直接用于每次查询。低随机性旋转基数树只用它生成节点初始化参数  $b_s$ ；由于新内部节点只在对应子树规模达到  $\text{polylog}(n)$  后创建，初始化成本可以分摊到此前的插入操作中。

由此得到的低随机性旋转基数树使用线性空间，可存储至多  $n$  个  $\Theta(\log n)$  位键值对，并以  $O(\log n \log \log n)$  个随机比特保证每次操作以  $1 - 1/\text{poly}(n)$  的概率在常数时间内完成。

## 6 空间效率

除了操作时间和失败概率外，空间效率也是字典结构中的重要指标。对于从全集  $U$  中存储  $n$  个键的集合，任意字典都需要至少

$$B(|U|, n) = \log \binom{|U|}{n} \quad (6)$$

比特的信息量。因此，简洁字典 (succinct dictionary) 的目标是将空间控制在接近该信息论下界的范围内，通常写作  $(1 + o(1))B(|U|, n)$ 。

Kuszmaul 进一步给出一种黑盒转换，使线性空间字典在基本保留原概率保证的同时，将空间降至

$$B(|U|, n) + O\left(\frac{n \log n}{\log \log n}\right) \quad (7)$$

比特<sup>[1]</sup>。该转换建立在 Raman 和 Rao 的简洁动态字典方法之上<sup>[4]</sup>，并把低随机性旋转基数树作为关键组件。

这一结果把论文的讨论从失败概率和随机比特扩展到空间冗余：增强旋转基数树优化失败概率，低随机性旋转基数树压缩随机性，而黑盒转换使二者能够进一步接近信息论空间下界。

## 7 结构关系与讨论

三个主要结构共享“旋转并叠加稀疏节点数组”这一机制，但优化目标不同。

旋转基数树建立线性空间的随机旋转框架，将传统哈希函数中的随机性转移到数据结构内部。

增强旋转基数树在基础结构上引入溢出球和辅助树，并用有界差分方法控制溢出规模，重点优化失败概率。

低随机性旋转基数树则通过减少内部节点、拆分映射参数并使用伪随机负载均衡工具，在保持  $1/\text{poly}(n)$  失败概率的同时，将随机比特数量压缩到  $O(\log n \log \log n)$ 。

因此，三种结构并非彼此独立的方案，而是同一随机旋转机制在失败概率、随机比特和空间效率三个目标上的展开。

**Table 1** 三类旋转树结构的目标与保证

| 结构        | 侧重点与结果                               |
|-----------|--------------------------------------|
| 旋转基数树     | 线性空间；失败概率为 $1/n^{\text{polylog}(n)}$ |
| 增强旋转基数树   | 线性空间；失败概率为 $O(1/n^{1-\epsilon})$     |
| 低随机性旋转基数树 | 线性空间；随机比特为 $O(\log n \log \log n)$   |

## 8 方法评价

该工作的主要贡献是改变随机化字典中随机性的承载方式。传统方法依靠哈希函数产生键到桶的随机映射；旋转基数树则随机旋转节点数组，使负载均衡问题转化为稀疏数组叠加后的桶负载分析。这一转化避开了显式哈希函数族对失败概率的直接限制。

低随机性旋转基数树还说明，求值时间超过常数的伪随机工具仍可参与常数时间数据结构的构造。其关键不是在每次操作中计算逐渐增强独立性哈希函数，而是仅在节点初始化时生成参数，再利用此前积累的插入操作分摊求值成本。

这些结果也有明确的适用边界。主体分析针对  $\Theta(\log n)$  位键值和相应的 word-RAM 模型，并采用操作序列独立于随机比特的遗忘型对手假设。动态融合节点、辅助树和多类伪随机工具还增加了实现复杂度。因此，该工作的价值主要在于推进概率界限并提供新的随机化设计路径，而非直接替代工程中的通用哈希表。

## 9 结论

本文梳理了 Kuszmaul 的无哈希函数哈希表构造。旋转基数树通过随机旋转并叠加节点数组获得线性空间；增强旋转基数树利用溢出球和 McDiarmid 不等式将失败概率降至  $O(1/n^{1-\epsilon})$ ；低随机性旋转基数树通过分组映射和摊销初始化，把随机比特压缩到  $O(\log n \log \log n)$ 。黑盒简洁化转换进一步使这些概率保证能够与接近信息论下界的空间开销结合。

这些结构表明，改进哈希表的概率保证并不只有增强哈希函数一条路径。将随机性嵌入数据结构本身，同样能够在操作时间、失败概率、随机比特和空间效率之间建立可分析的权衡。

## 参 考 文 献

- [1] KUSZMAUL W. A hash table without hash functions, and how to get the most out of your random bits[C]//2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS). IEEE, 2022: 991-1001.
- [2] PATRASCU M, THORUP M. Dynamic integer sets with optimal rank, select, and predecessor search[C]//2014 IEEE 55th Annual Symposium on Foundations of Computer Science (FOCS). IEEE, 2014: 166-175.
- [3] MCDIARMID C. On the method of bounded differences[J]. Surveys in Combinatorics, 1989, 141: 148-188.
- [4] RAMAN R, RAO S S. Succinct dynamic dictionaries and trees[C]//Automata, Languages and Programming. Springer, 2003: 357-368.